



ScholarNest

Spark Azure Databricks

Databricks Spark Certification and beyond

Instructor: Prashant Kumar Pandey



Azure
Databricks



Absolute Beginner to Specialization in Apache Spark and Azure Databricks



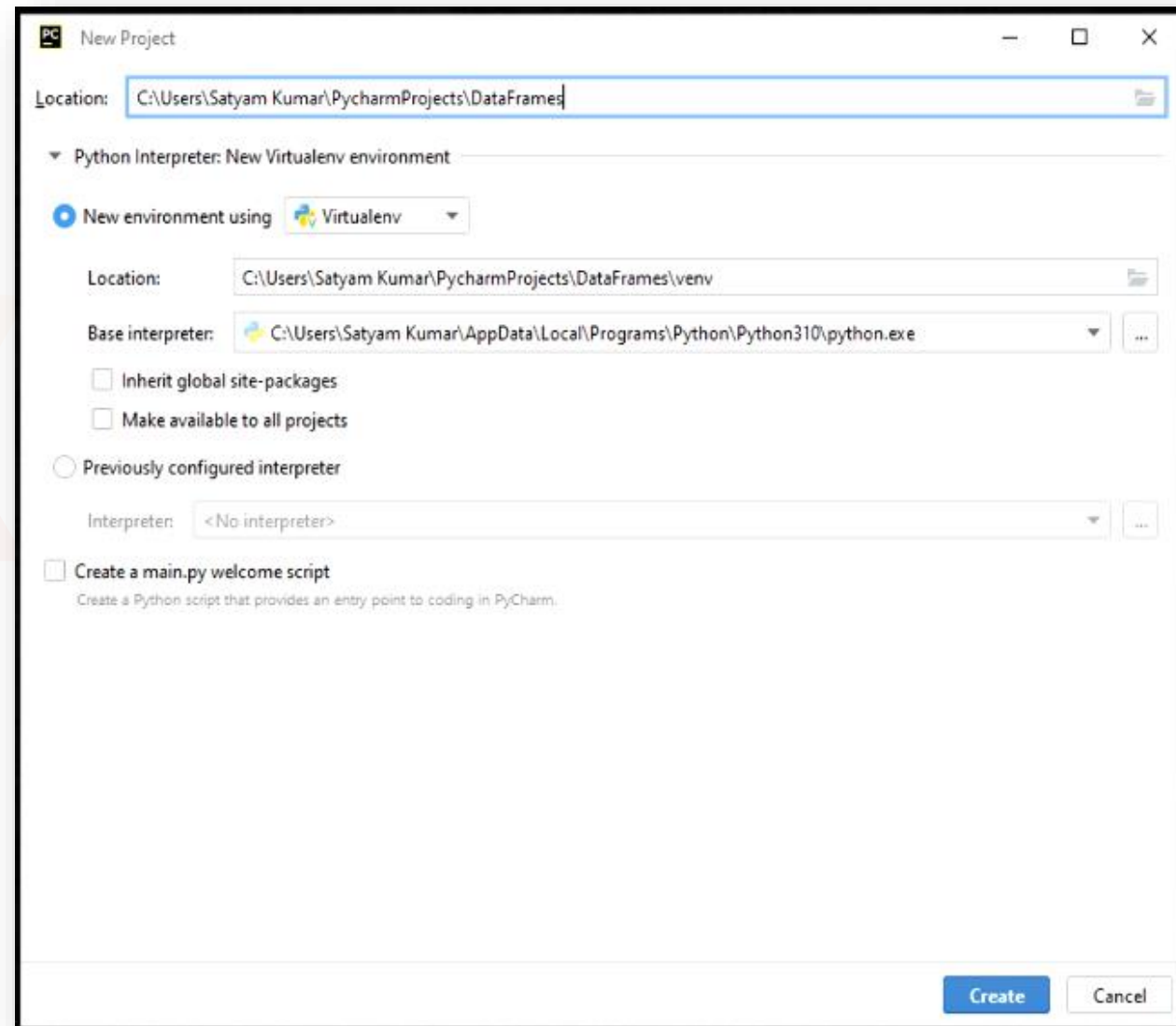
Creating Spark Session

Assume you are given a task to create a small Spark job to do the following:

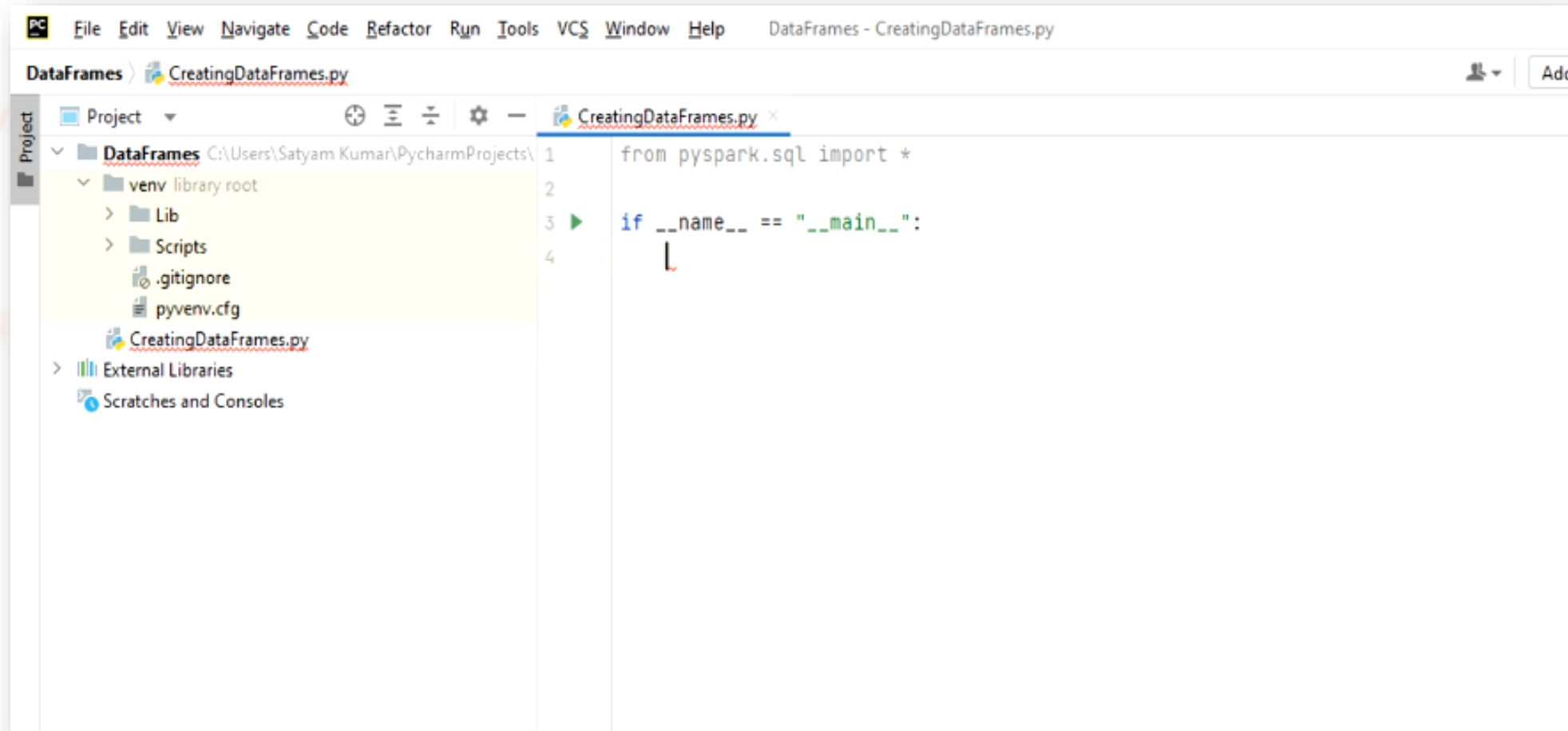
Requirement

1. Read a CSV data file
2. Perform some standard data validation and data type correction to prepare it for analysis
3. Save the valid data using the parquet data file format

Start your PyCharm IDE and create a new project. Give a name to your project and create it. **(Reference : DataFrames)**



Create a new Python file in your project. We will be using the PySpark SQL package so let me import the package. I am starting a new Spark project, so I need an entry point for my application. So I have defined the main method as shown below.



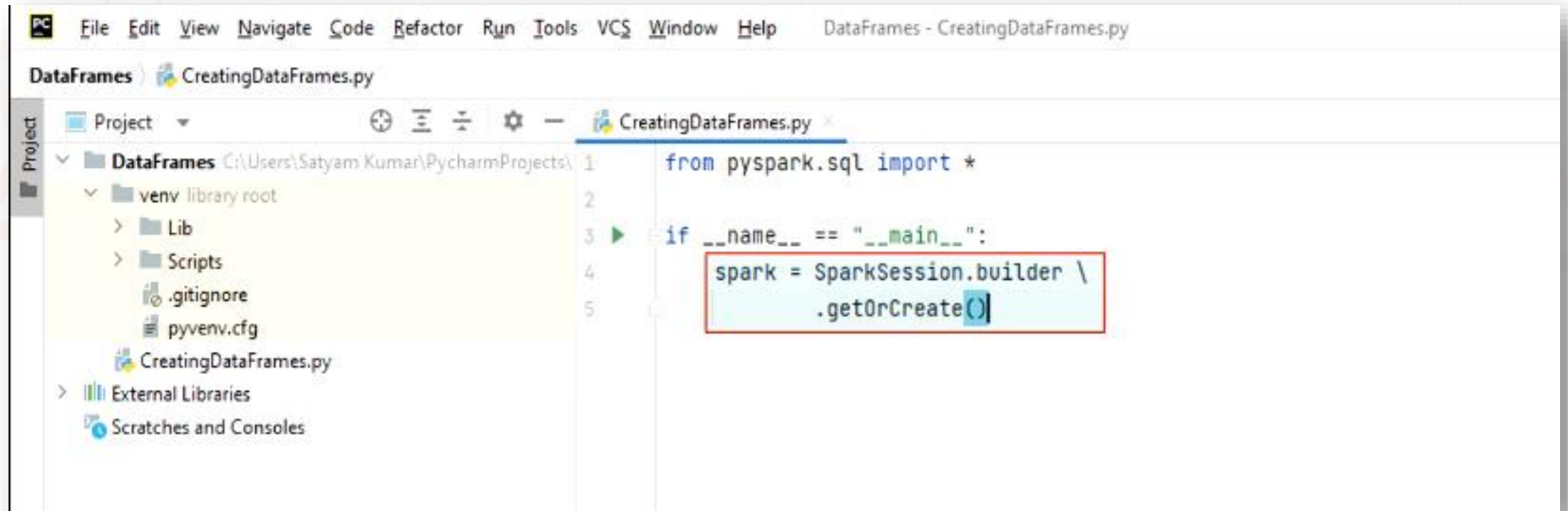
The first requirement is to read a CSV data file. How will you do it? Here it is. We used this code to read a CSV file and create a Spark DataFrame. So we can use the same code template here.



The screenshot shows the PyCharm IDE interface. The left sidebar displays the project structure for 'DataFrames', including a 'venv' directory and 'CreatingDataFrames.py'. The main editor window shows the code for 'CreatingDataFrames.py'. The code imports pyspark.sql and defines a main function that reads a CSV file into a Spark DataFrame. A red box highlights the DataFrame creation code.

```
1 from pyspark.sql import *
2
3 if __name__ == "__main__":
4     raw_fire_df = spark.read \
5         .format("CSV") \
6         .option("header", "true") \
7         .option("inferSchema", "true") \
8         .load("/databricks-datasets/learning-spark-v2/sf-fire-calls.csv")
```

But we have a small problem. We do not have the Spark Session variable. I created the earlier example in a Databricks Notebook. The notebook environment automatically creates a Spark session object and makes it available for us. But when you are creating a Spark program in a Python file, you must create the SparkSession yourself as shown below.

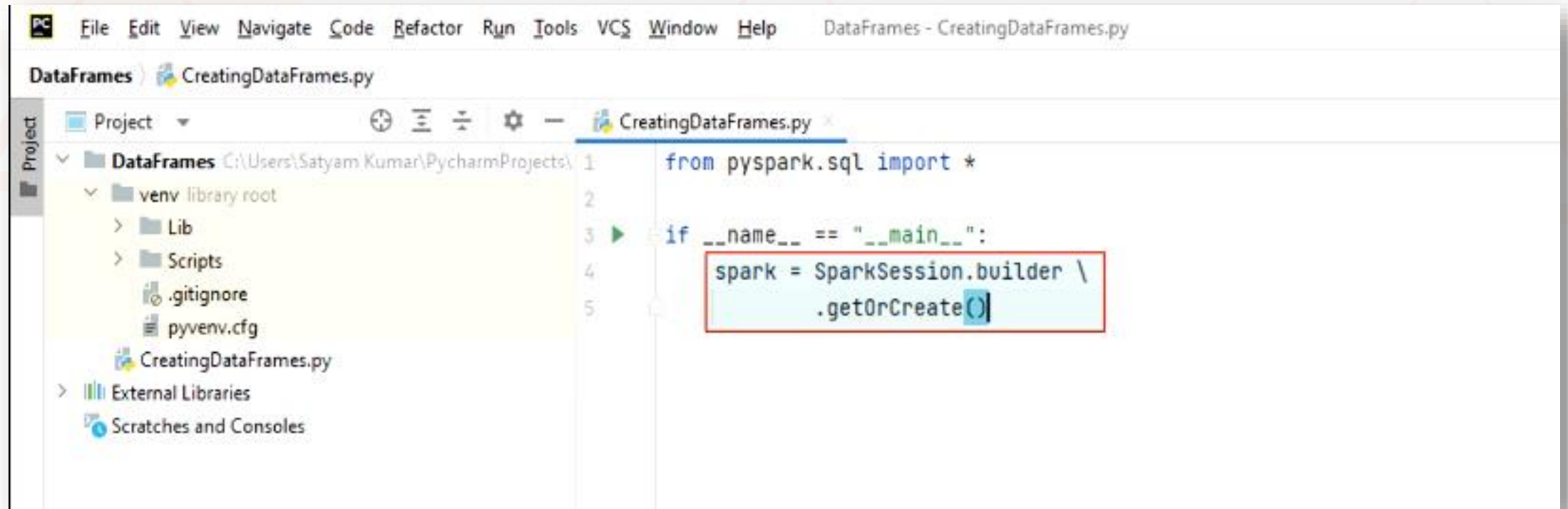


The screenshot shows the PyCharm IDE interface. The top menu bar includes File, Edit, View, Navigate, Code, Refactor, Run, Tools, VCS, Window, and Help. The title bar indicates the file is 'DataFrames - CreatingDataFrames.py'. The left sidebar shows the project structure for 'DataFrames' at 'C:\Users\Satyam Kumar\PycharmProjects\'. It includes a 'venv' library root with 'Lib' and 'Scripts' folders, a '.gitignore' file, a 'pyvenv.cfg' file, and the 'CreatingDataFrames.py' file. The main editor window displays the code in 'CreatingDataFrames.py' with line numbers 1 through 5. The code is as follows:

```
1 from pyspark.sql import *
2
3 if __name__ == "__main__":
4     spark = SparkSession.builder \
5         .getOrCreate()
```

The code on lines 4 and 5 is highlighted with a red rectangular box, indicating the manual creation of the SparkSession object.

But we have a small problem. We do not have the Spark Session variable. I created the earlier example in a Databricks Notebook. The notebook environment automatically creates a Spark session object and makes it available for us. But when you are creating a Spark program in a Python file, you must create the SparkSession yourself as shown below. The code shown here is the bare minimum to start writing the Spark program in a python file.



The screenshot shows the PyCharm IDE interface. The top menu bar includes File, Edit, View, Navigate, Code, Refactor, Run, Tools, VCS, Window, and Help. The title bar indicates the file is 'DataFrames - CreatingDataFrames.py'. The left sidebar shows the project structure for 'DataFrames' at 'C:\Users\Satyam Kumar\PycharmProjects\'. It includes a 'venv' library root with 'Lib' and 'Scripts' folders, a '.gitignore' file, a 'pyvenv.cfg' file, and the 'CreatingDataFrames.py' file. The main editor window displays the following Python code:

```
1 from pyspark.sql import *
2
3 if __name__ == "__main__":
4     spark = SparkSession.builder \
5         .getOrCreate()
```

The code on lines 4 and 5 is highlighted with a red rectangular box.

Spark Session is your entry point for Spark programming, and it offers you the following attributes and methods.

SparkSession

Attributes

<code>builder</code>	A class attribute having a <code>Builder</code> to construct <code>SparkSession</code> instances.
<code>read</code>	Returns a <code>DataFrameReader</code> that can be used to read data in as a <code>DataFrame</code> .
<code>readStream</code>	Returns a <code>DataStreamReader</code> that can be used to read data streams as a streaming <code>DataFrame</code> .
<code>catalog</code>	Interface through which the user may create, drop, alter or query underlying databases, tables, functions, etc.
<code>conf</code>	Runtime configuration interface for Spark.
<code>udf</code>	Returns a <code>UDFRegistration</code> for UDF registration.
<code>sparkContext</code>	Returns the underlying <code>SparkContext</code> .
<code>version</code>	The version of Spark on which this application is running.

Methods

<code>createDataFrame(data[, schema, ...])</code>	Creates a <code>DataFrame</code> from an <code>RDD</code> , a list or a <code>pandas.DataFrame</code> .
<code>getActiveSession()</code>	Returns the active <code>SparkSession</code> for the current thread, returned by the builder.
<code>newSession()</code>	Returns a new <code>SparkSession</code> as new session, that has separate <code>SQLConf</code> , registered temporary views and UDFs, but shared <code>SparkContext</code> and table cache.
<code>range(start[, end, step, numPartitions])</code>	Create a <code>DataFrame</code> with single <code>pyspark.sql.types.LongType</code> column named <code>id</code> , containing elements in a range from <code>start</code> to <code>end</code> (exclusive) with step value <code>step</code> .
<code>sql(sqlQuery)</code>	Returns a <code>DataFrame</code> representing the result of the given query.
<code>stop()</code>	Stop the underlying <code>SparkContext</code> .
<code>table(tableName)</code>	Returns the specified table as a <code>DataFrame</code> .

Let's focus on the attributes for now. The first one is the builder. Once you have the session builder, you can use builder methods to create and configure your spark session. And here is the list of session builder methods. So we have five session builder methods. The appName allows you to set a unique name for your Spark application.

SparkSession

Attributes	
builder	A class attribute having a Builder to construct SparkSession instances.
read	Returns a DataFrameReader that can be used to read data in as a DataFrame.
readStream	Returns a DataStreamReader that can be used to read data streams as a streaming DataFrame.
catalog	Interface through which the user may create, drop, alter or query underlying databases, tables, functions, etc.
conf	Runtime configuration interface for Spark.
udf	Returns a UDFRegistration for UDF registration.
sparkContext	Returns the underlying SparkContext.
version	The version of Spark on which this application is running.

Builder Methods

SparkSession.builder.appName(name)	Sets a name for the application, which will be shown in the Spark web UI.
SparkSession.builder.master(master)	Sets the Spark master URL to connect to, such as "local" to run locally, "local[4]" to run locally with 4 cores, or "spark://master:7077" to run on a Spark standalone cluster.
SparkSession.builder.config([key, value, conf])	Sets a config option.
SparkSession.builder.enableHiveSupport()	Enables Hive support, including connectivity to a persistent Hive metastore, support for Hive SerDes, and Hive user-defined functions.
SparkSession.builder.getOrCreate()	Gets an existing SparkSession or, if there is no existing one, creates a new one based on the options set in this builder.

The next one is to set the location of the master node of your Spark cluster. I do not recommend using this method at all. You should never set the master from your code.

Why? Because you will run your code on your local machine or in the development cluster. Then you want to run the same application in a testing cluster. Finally, the code goes to the production cluster.

The master node location is different everywhere. So we do not hardcode it in the application. Instead, we set the master node location from outside the application code.

The config method allows you to configure your spark session.

The next one is to enable Hive support in your Spark application. Enabling Hive support allows you to access Hive database tables from Spark. You should enable this option when your target is the Hadoop cluster. But this option is not meaningful in the Databricks Cloud environment. Because you do not have Hadoop and Hive in Databricks Cloud.

The last and the most important one is to getOrCreate a Spark session.

Spark application must have only one active Spark session at any given point. So the getOrCreate API protects you from incorrectly creating two sessions. If you already created a SparkSession earlier, the getOrCreate will return the same session; else, it will create a new session. So even if you call the getOrCreate method more than once, you will get the same Spark session. Make sense?

We have quite a few other attributes of Spark Session. The read and the readStream are the most important. They give you a DataFrameReader and open your gateway to the DataFrame API.

The catalog is your Spark metadata interface.
The conf is your spark runtime configuration interface.
The UDF here gives you a UDFRegistration interface.

The last one is the SparkContext.

A SparkContext represents the connection to a Spark cluster and can use to create RDD and broadcast variables on the cluster.

Memorize this definition.

The SparkContext offers you two functionalities.

1. It holds and manages a connection to your Spark cluster
2. It is your gateway to Spark Core APIs

So Spark session also offers you two functionalities.

1. It is your gateway to Spark DataFrame APIs
2. It holds the SparkContext.

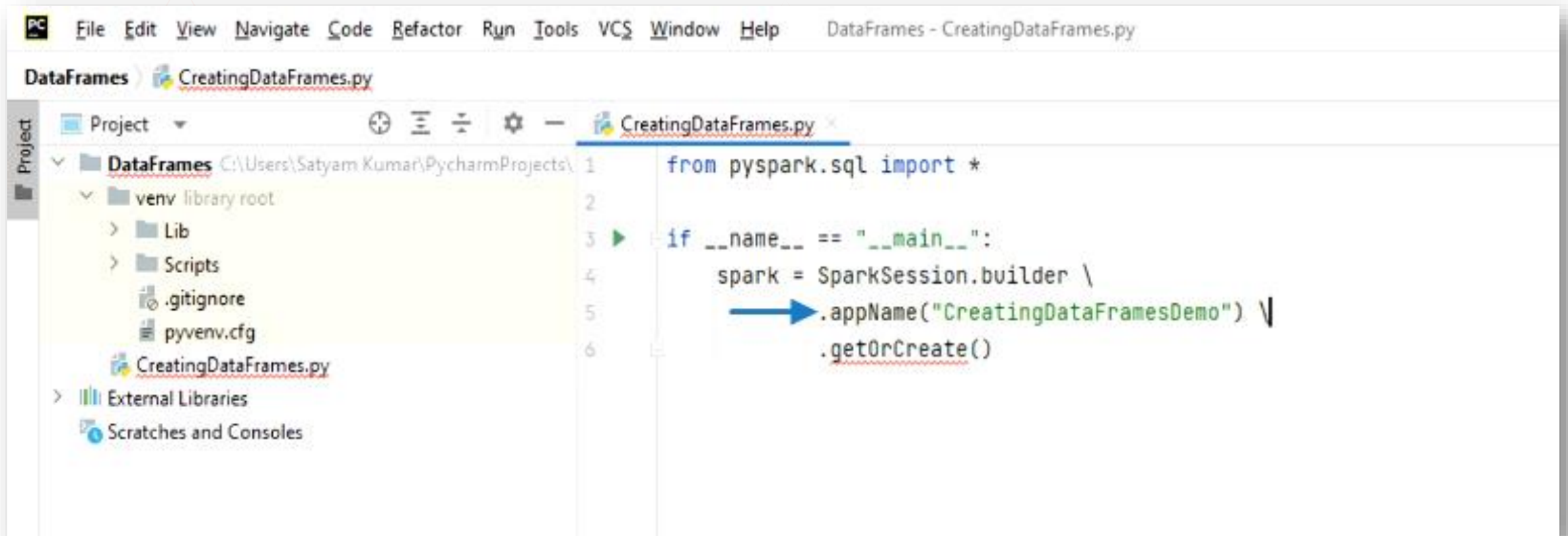
Now you can combine the SparkContext into SparkSession.

1. It is your gateway to Spark DataFrame APIs
2. It holds the SparkContext.
 1. It holds and manages a connection to your Spark cluster
 2. It is your gateway to Spark Core APIs

You learned to create a SparkSession.

It is the bare minimum code. But I recommend setting a unique application name.

Simply add a line just before the `getOrCreate` and use Session Builder's `appName()` method to set a Unique application name.



```
1 from pyspark.sql import *
2
3 if __name__ == "__main__":
4     spark = SparkSession.builder \
5         → .appName("CreatingDataFramesDemo") \
6         .getOrCreate()
```

Summary



You always need a SparkSession.



Notebooks and command-line tools automatically create a SparkSession for you



But you must create it yourself in all other environments.



We learned the bare minimum code for creating a Spark Session



I also talked about the session builder and the SparkContext



Thank You
ScholarNest Technologies Pvt Ltd.
www.scholarnest.com